Windows + Docker + JupyterLab + PySpark

Complete Step-by-Step Guide for Running the Healthcare Analytics Notebook

This guide shows how to run the Healthcare Patient Analytics notebook on Windows using Docker Desktop, WSL 2, JupyterLab and PySpark. It also fixes the main performance issue in the notebook: creating one million Python records before handing them to Spark.

Recommended architecture: Windows → Docker Desktop → WSL 2 → Jupyter pyspark-notebook → JupyterLab → Spark-native data generation.

1. Why the Current Notebook Is Slow

The original notebook creates a Python list containing 1,000,000 tuples and then converts that list into a Spark DataFrame. This can use substantial RAM and adds Python-to-JVM serialization overhead. Docker gives you a consistent environment, but Docker alone does not make this code faster. The important optimization is to generate the large dataset directly with Spark.

2. What You Need

• Windows 10/11 with hardware virtualization enabled.

• Docker Desktop for Windows.

• WSL 2 enabled.

• Preferably 8 GB or more system RAM.

• The healthcare_pyspark.ipynb notebook.

• Chrome or Edge.

3. Install Docker Desktop

Official guide: https://docs.docker.com/desktop/setup/install/windows-install/

• Download Docker Desktop for Windows.

• Run Docker Desktop Installer.exe.

• Use the WSL 2 backend when offered.

• Complete installation and start Docker Desktop.

• Wait until Docker Desktop is running.

Docker's current Windows documentation supports WSL 2 and lists it as the default/recommended backend for most Windows users.

4. Install or Update WSL 2

Open PowerShell as Administrator:

```
wsl --version
wsl --update
```

If WSL is not installed:

```
wsl --install
```

Restart Windows if requested, then verify:

```
wsl --status
wsl -l -v
```

5. Configure Docker to Use WSL 2

• Open Docker Desktop.

• Settings → General → confirm 'Use WSL 2 based engine'.

• Settings → Resources → WSL Integration.

• Enable your WSL distribution if you use one.

• Click Apply & Restart.

6. Verify Docker

```
docker --version
docker run hello-world
```

If 'Hello from Docker!' appears, Docker is working.

7. Create Your Project Folder

```
C:\pyspark-healthcare
```

Place your notebook here:

```
C:\pyspark-healthcare\healthcare_pyspark.ipynb
```

8. Download the PySpark Jupyter Image

The Jupyter Docker Stacks project provides a pyspark-notebook image containing Python support for Apache Spark, Spark/Hadoop binaries and related data-science packages.

```
docker pull quay.io/jupyter/pyspark-notebook:latest
```

The first pull can take time because the image is large. Later runs reuse the local image.

9. Start JupyterLab + PySpark

```
cd C:\pyspark-healthcare
docker run --rm -it -p 8888:8888 -p 4040:4040 -v "${PWD}:/home/jovyan/work" quay.io/jupyter/pyspark-noteboo
```

The volume maps your Windows folder to /home/jovyan/work inside the container. Port 8888 is JupyterLab; port 4040 is the Spark UI.

10. Open JupyterLab

• Copy the Jupyter URL shown in PowerShell, usually beginning with http://127.0.0.1:8888/lab?token=...

• Paste it into Chrome or Edge.

• Open the work folder.

• Open healthcare_pyspark.ipynb.

11. Verify PySpark

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
         .appName("HealthcareTest")
         .master("local[*]")
         .getOrCreate())

print("Spark version:", spark.version)
```

If the Spark version prints, the environment is ready.

12. IMPORTANT: Replace the Slow Data Generator

Do not immediately run the original million-row Python loop. Replace it with Spark-native generation. This lets Spark manage the records instead of building a huge Python list first.

13. Spark-Native 1 Million Record Generator

```
from pyspark.sql import functions as F

NUM_RECORDS = 1_000_000

df = (spark.range(NUM_RECORDS)
      .withColumnRenamed("id", "visit_id")
      .withColumn("patient_id", (F.rand(seed=42)*200000+1).cast("int"))
      .withColumn("age", (F.rand(seed=43)*89+1).cast("int"))
      .withColumn("gender", F.when(F.rand(seed=44)<0.48,"Male")
                           .when(F.rand(seed=45)<0.96,"Female")
                           .otherwise("Other")))
```

14. Add Healthcare Columns

```
df = (df
 .withColumn("condition", F.element_at(F.array(
     F.lit("Diabetes"),F.lit("Hypertension"),F.lit("Heart Disease"),F.lit("Asthma"),
     F.lit("Cancer"),F.lit("Kidney Disease"),F.lit("Arthritis"),F.lit("Migraine")
 ), (F.rand(seed=50)*8+1).cast("int")))
 .withColumn("hospital", F.element_at(F.array(
     F.lit("City Hospital"),F.lit("Apollo Care"),F.lit("Metro Hospital"),
     F.lit("Green Valley Hospital"),F.lit("Sunrise Medical Center")
 ), (F.rand(seed=51)*5+1).cast("int")))
 .withColumn("department", F.element_at(F.array(
     F.lit("Cardiology"),F.lit("General Medicine"),F.lit("Neurology"),
     F.lit("Orthopedics"),F.lit("Oncology"),F.lit("Pulmonology"),F.lit("Nephrology")
 ), (F.rand(seed=52)*7+1).cast("int")))
 .withColumn("length_of_stay", (F.rand(seed=53)*20+1).cast("int"))
 .withColumn("treatment_cost", F.round(F.rand(seed=54)*145000+5000,2))
 .withColumn("admission_date", F.date_add(F.to_date(F.lit("2025-01-01")),
                               (F.rand(seed=55)*365).cast("int")))
)
```

15. Verify the Dataset

```
df.show(10, truncate=False)
print("Records:", df.count())
```

Expected: 1,000,000 records.

16. Continue With the Healthcare Analysis

```
condition_analysis = (df.groupBy("condition")
 .agg(F.count("*").alias("visits"),
      F.countDistinct("patient_id").alias("patients"),
      F.round(F.avg("treatment_cost"),2).alias("avg_treatment_cost"),
      F.round(F.avg("length_of_stay"),2).alias("avg_length_of_stay"))
 .orderBy(F.desc("patients")))

condition_analysis.show()
```

17. Do Not Convert the Full Dataset to Pandas

Avoid df.toPandas() on the complete 1-million-row dataset. Convert only small aggregated results:

```
result = df.groupBy("condition").agg(F.count("*").alias("patients"))
result_pd = result.toPandas()
```

18. Open the Spark Web UI

```
http://localhost:4040
```

Use the Spark UI to demonstrate Jobs, Stages, SQL and execution activity to learners.

19. Recommended Docker Memory

System RAM	Suggested Docker memory	Training dataset
8 GB	Up to ~4 GB	100K–500K first
16 GB	~8 GB	1 million
32 GB	~12–16 GB	1–10 million

In Docker Desktop, review Settings → Resources. Do not allocate essentially all system RAM to Docker; Windows and the browser also need memory.

20. Best Training Sequence

• Start with NUM_RECORDS = 100_000.

• Confirm cleaning and aggregations work.

• Open Spark UI and explain Jobs/Stages.

• Increase to 1_000_000.

• Repeat the analysis.

• Explain why Spark becomes useful as data volume increases.

• Use Pandas only for small aggregated outputs.

21. Common Problems and Fixes

Problem	Fix
docker not recognized	Start Docker Desktop and reopen PowerShell.
WSL version missing	Run wsl --update as Administrator and restart if requested.
Docker daemon not running	Open Docker Desktop and wait until it is running.
Port 8888 busy	Use -p 8889:8888 and open http://localhost:8889.
Port 4040 busy	Use another host port such as -p 4041:4040.
Notebook not visible	Confirm the notebook is inside C:\pyspark-healthcare and the volume mapping is correct.
Kernel becomes slow	Reduce NUM_RECORDS and avoid full toPandas().
Container stops	Check Docker Desktop RAM/resource settings.
First Spark startup is slow	Some JVM/Spark initialization delay is normal; reuse the same Spark session.

22. Stop the Environment

Save the notebook. In the PowerShell window running the container, press Ctrl+C. Because the command used --rm, the container is removed after it stops. Your notebook remains in the Windows folder because that folder is mounted into the container.

23. Quick Command Reference

```
wsl --version
wsl --update
wsl --status
wsl -l -v
docker --version
docker run hello-world
docker pull quay.io/jupyter/pyspark-notebook:latest
```

```
cd C:\pyspark-healthcare
docker run --rm -it -p 8888:8888 -p 4040:4040 -v "${PWD}:/home/jovyan/work" quay.io/jupyter/pyspark-noteboo
```

24. Official References

• Docker Desktop for Windows: https://docs.docker.com/desktop/setup/install/windows-install/

• Docker Desktop WSL 2: https://docs.docker.com/desktop/features/wsl/

• Jupyter Docker Stacks - Running containers: https://jupyter-docker-stacks.readthedocs.io/en/latest/using/running.html

• Jupyter Docker Stacks - Image selection: https://jupyter-docker-stacks.readthedocs.io/en/latest/using/selecting.html

• Jupyter Docker Stacks - Spark specifics: https://jupyter-docker-stacks.readthedocs.io/en/latest/using/specifics.html

Key takeaway: Docker gives you a consistent PySpark/Jupyter environment, but the biggest performance improvement is replacing the Python 1-million-record list with Spark-native data generation.